Projektuppgift

Frontend-baserad webbutveckling

Applikation

Anna Zach

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Anna Zach, anza2501@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Projektuppgift
Anna Zach 2026-03-19

Sammanfattning

Målet med detta projekt har varit att utveckla en webbapplikation som kombinerar data från flera externa webbtjänster för att ge användaren ett användbart verktyg vid reseplanering. Applikationen hämtar och presenterar information om väder, karta och sevärdheter för en vald stad. Syftet har varit att skapa en tydlig och användbar applikation samt att tillämpa kunskaper inom modern webbutveckling.

Applikationen är utvecklad med JavaScript utan frontend-ramverk, tillsammans med byggverktyget Vite för att hantera utvecklingsmiljön och produktion. Data hämtas via API-anrop och presenteras dynamiskt i gränssnittet, vilket ger en applikationsliknande upplevelse. SASS/SCSS används för styling och CSS-animationer används för att göra sidan mer levande och interaktiv. Animationerna inkluderar väderspecifika ikoner för regn, sol och moln samt ett flygplan.

Användaren kan söka efter en stad och få upp en karta med markör, väderprognos för de kommande dagarna samt en lista över sevärdheter. Applikationen fungerar på både dator och mobil, vilket visar att det är möjligt att kombinera flera datakällor på ett effektivt sätt. Resultaten visar att webbapplikationen uppfyller sina mål och ger en god användarupplevelse.

Vidare finns stora möjligheter till utveckling, till exempel att visa bilder på sevärdheter, lägga in länkar till flyg, hotell och restauranger, låta användaren spara planerade resor och se genomförda resor som en personlig minnesbok. Dessa funktioner skulle ge användaren ett komplett verktyg för reseplanering och inspiration.

Innehållsförteckning

Sammanfattning **iii**

Innehållsförteckning **iv**

Terminologi **v**

1 Introduktion / Inledning **1**
1.1 Bakgrund och problemmotivering 1
1.2 Syfte 1
1.3 Avgränsningar 1
1.4 Mål 1
1.5 Översikt 1
1.6 Författarens bidrag 1

2 Teori / Bakgrundsmaterial **2**

3 Metod **3**

4 Konstruktion **4**

5 Resultat **7**

6 Slutsatser / Analys / Diskussion **8**

6.1 Analys **8**

6.2 Egen reflektion **8**

Källförteckning **9**

Bilaga A: Dokumentation av egenutvecklad programkod **10**
Figur A1 10
Figur A2 10

Terminologi

I detta avsnitt förklaras de viktigaste begreppen, förkortningarna och variablerna som används i rapporten.

API (Application Programming Interface) – Ett gränssnitt som låter olika program eller system kommunicera och utbyta data [1].

JSON (JavaScript Object Notation) – Ett lättläst textformat som används för att strukturera och överföra data mellan system [2].

Asynkron programmering – Teknik som gör att program kan hämta data i bakgrunden utan att sidan fryser eller väntar, vilket ger bättre användarupplevelse [3].

Koordinater – Latitud och longitud som anger en exakt plats på jorden. Används i kartor och platsbaserade applikationer [4].

Vite – Ett modernt byggverktyg för webbprojekt som optimerar och hanterar kod under utveckling [5].

Leaflet – Ett JavaScript-bibliotek som används för interaktiva kartor, inklusive markeringar och rutter [6].

SCSS (Sassy CSS) – Ett förlängningsspråk för CSS som gör det enklare att strukturera, återanvända och underhålla stilar [7].

1 Introduktion / Inledning

1.1 Bakgrund och problemmotivering

Många människor använder idag flera olika webbplatser eller appar för att planera resor. Man behöver kontrollera väder, hitta sevärdheter och orientera sig på en karta, ofta via olika tjänster. Detta kan upplevas som splittrat och ineffektivt.

1.2 Övergripande syfte

Projektet syftar till att förenkla denna process genom att samla relevant information i en och samma webbapplikation. Genom att kombinera flera API:er kan användaren få en mer komplett bild av en destination utan att byta mellan olika tjänster.

1.3 Avgränsningar

Projektet är avgränsat till att använda ett begränsat antal API:er och fokuserar på grundläggande funktionalitet. Applikationen innehåller inte funktioner som användarkonton, bokningar eller sparade resor.

1.4 Mål

Målet med projektet är att skapa en fungerande webbapplikation där användaren kan söka efter en stad och få upp relevant information i form av väderprognos, karta och sevärdheter. Applikationen ska vara responsiv och ge en tydlig och användbar upplevelse.

1.5 Översikt

Rapporten beskriver först relevant teori, därefter metod och genomförande. Sedan presenteras resultatet och avslutningsvis en analys av projektet.

1.6 Författarens bidrag

Arbetet har genomförts individuellt och omfattar design, implementation, API-integration samt dokumentation.

2 Teori / Bakgrundsmaterial

Projektet bygger på flera centrala tekniker inom modern webbutveckling. En viktig del är användningen av API:er (Application Programming Interfaces)[1], vilket gör det möjligt för olika system att kommunicera med varandra och utbyta data. I detta projekt används JSON-baserade API:er för att hämta information om väder, koordinater och sevärdheter. JSON är ett vanligt dataformat som används för att strukturera och överföra data mellan system [2].

För att hantera dessa data används asynkron programmering, vilket innebär att applikationen kan hämta data utan att blockera användargränssnittet. Detta görs med hjälp av funktioner som async och await, vilket gör koden mer effektiv och lättläst [3].

Ett byggverktyg, Vite, används för att hantera utvecklingsmiljön och skapa en optimerad version av applikationen för produktion. Detta innebär att filer minskas och struktureras för att förbättra prestanda [5].

För kartfunktionalitet används Leaflet, vilket är ett JavaScript-bibliotek för att skapa interaktiva kartor i webbläsaren. I projektet används detta för att visa en karta med en markör som representerar den valda platsen [6].

Stylingen är gjord med SCSS [7], vilket är en vidareutveckling av CSS som gör det möjligt att strukturera kod mer effektivt. Slutligen används CSS-animationer för att skapa en mer levande och interaktiv användarupplevelse.

3 Metod

Arbetet började med skisser i Figma för att planera layout och användargränssnitt. Därefter genomfördes implementering av basfunktioner med JavaScript, inklusive API-hämtning för väderdata, koordinater och sevärdheter.

Varje funktion testades individuellt innan de integrerades, och responsivitet kontrollerades på både dator och mobil. Data hämtades asynkront för att inte blockera sidan, och animationer implementerades för bättre användarupplevelse.

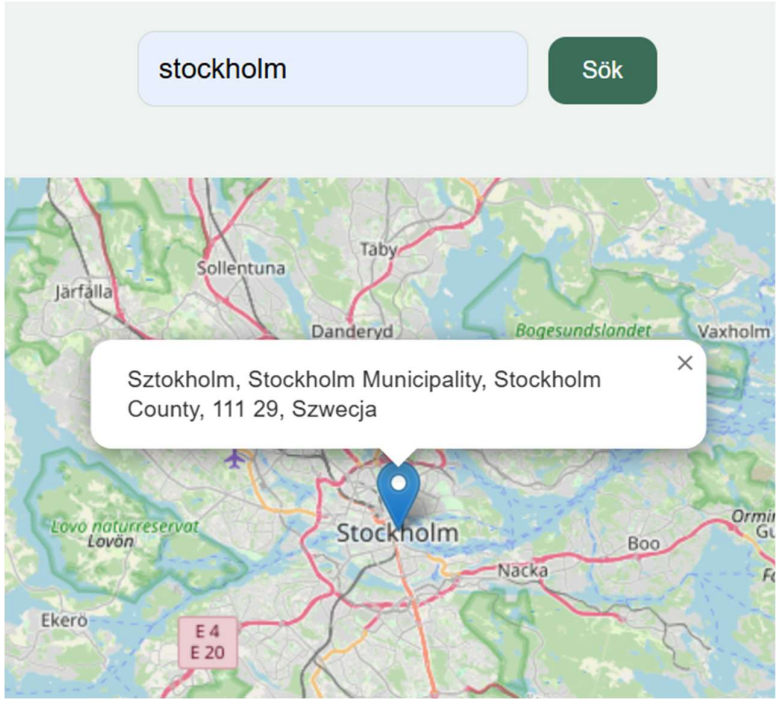
En iterativ metod användes: kravanalys → skisser → implementering → test → förbättring. Problem analyserades för att säkerställa systematisk uveckling.

4 Konstruktion / Lösningsalternativ

Applikationen är uppbyggd så att användaren interagerar med ett sökfält där en stad anges. När en sökning görs hämtas stadens koordinater, vilka används för att visa en karta med en markör på rätt plats samt för att hämta väderdata och information om sevärdheter.

Kartan visualiserar tydligt den valda platsen och uppdateras dynamiskt vid varje sökning (se figur 1).

Figur 1: Karta med markör som visar vald stad.



Väderinformationen presenteras i form av kort som visar temperatur och väderförhållanden för flera dagar framåt (se figur 2).

Figur 2: Exempel på väderprognos som visas i applikationen.

Sevärdheter presenteras som en lista med namn, vilket ger användaren en snabb överblick över vad som finns att göra i området (se figur 3).

Figur 3: Lista över sevärdheter för vald stad.

För att göra sidan mer levande och interaktiv har CSS-animationer lagts till, exempelvis vädersymboler för sol, regn och moln, samt ett flygplan som rör sig över sidan. Animationerna fungerar som visuella effekter som inte tar över för mycket, men som förbättrar användarupplevelsen (se figur 4).

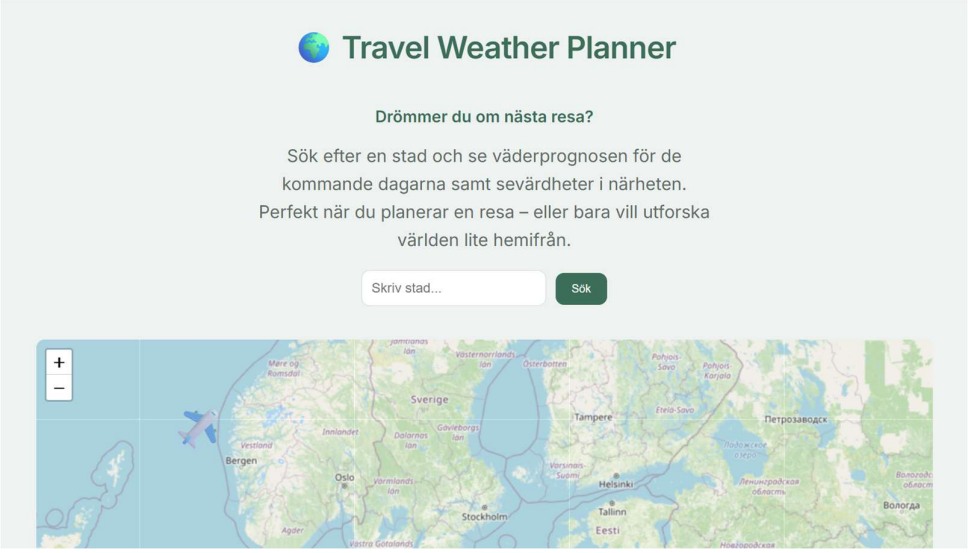
Figur 4: Exempel på animationer i applikationen.

Möjliga förbättringar och framtida utvecklingsområden inkluderar att visa bilder och länkar till sevärdheter samt att ge användaren möjlighet att boka biljetter till museer och andra attraktioner. Det skulle även vara önskvärt att kunna boka flyg, hotell och restaurangbesök direkt i applikationen. Vidare skulle användaren kunna spara sina planerade resor inklusive alla bokningar, och varför inte även registrera genomförda resor för att skapa en personlig minnesbok. Dessa funktioner skulle göra applikationen mer komplett och ge ett mer omfattande verktyg för reseplanering.

5 Resultat

Resultatet är en fungerande webbapplikation där användaren kan söka efter en stad och få upp karta, väderprognos och sevärdheter. Applikationen är responsiv och fungerar både på dator och mobil. (se figur 5).

Figur 5: Översikt av applikationens användargränssnitt.



Data uppdateras korrekt vid varje sökning, och designen är enkel men tydlig. CSS-animationer ger ett mer interaktivt intryck, och väderspecifika ikoner ger visuell feedback.

Projektet har gett en bra förståelse för hur man kan kombinera flera olika tekniker för att skapa en mer avancerad webbapplikation. En av de största utmaningarna har varit att få alla delar att fungera tillsammans, särskilt när det gäller att hantera data från flera API:er.

Det uppstod också vissa problem under utvecklingen, till exempel med visning av data och kartans markör, men dessa kunde lösas genom felsökning och justeringar i koden. Även mindre detaljer, som animationens beteende på olika skärmstorlekar, krävde viss finjustering.

En viktig lärdom från projektet är hur viktigt det är att testa kontinuerligt och arbeta stegvis. Genom att bygga upp funktionaliteten i mindre delar blev det lättare att identifiera problem.

6 Slutsatser

6.1 Analys

Projektet uppfyller sina mål och har gett en bra förståelse för hur man kan kombinera flera olika tekniker för att skapa en mer avancerad webbapplikation.

Möjligheter till vidare utveckling inkluderar integration av fler tjänster, förbättrad design, fler animationer och användarprofiler. Projektet har visat att det går att kombinera flera datakällor på ett effektivt sätt för att skapa en interaktiv och användbar webbapplikation.

6.2 Egen reflektion

Jag tyckte att detta var en väldigt rolig uppgift eftersom det kändes som att jag byggde något som faktiskt kan vara användbart i verkligheten. Det var också motiverande att se hur sidan växte fram steg för steg.

Jag märkte att jag gärna ville utveckla projektet vidare, vilket visar att det finns stor potential i idén. Om sidan skulle utvecklas vidare skulle det vara spännande att kunna boka flyg och restaurangbesök, samt spara planerade resor och få en personlig minnesbok över genomförda resor. Detta skulle göra applikationen ännu mer komplett och användbar.

Källförteckning

[1] MDN Web Docs, “APIs (Application Programming Interfaces)”,
https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction,
hämtad 2026-03-19

[2] Wikipedia, “JSON”,
https://en.wikipedia.org/wiki/JSON,
hämtad 2026-03-19

[3] Mozilla Developer Network, “Asynchronous programming”,
https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous
hämtad 2026-03-19

[4] Google Developers, “Latitude and Longitude”.
https://developers.google.com/maps/documentation/geocoding/overview, hämtad 2026-03-19

[5] Vite, “Getting Started”,
https://vitejs.dev/guide/,
hämtad 2026-03-19

[6] Leaflet, “Introduction”,
https://leafletjs.com/,
hämtad 2026-03-19

[7] Sass, “SCSS Syntax”,
https://sass-lang.com/guide,
hämtad 2026-03-19

Bilaga A: Dokumentation av egenutvecklad programkod

I denna bilaga presenteras koden som används i webbapplikationen.

Figur A1: Temperaturfunktion och huvudfunktion

```
/**
 * Returnerar CSS-klass för temperatur.
 * @param {number} temp - Temperatur i Celsius.
 * @returns {string} - 'hot', 'cold' eller 'mild'.
 */
function getTempClass(temp) {
  if (temp >= 25) return 'hot'
  if (temp <= 10) return 'cold'
  return 'mild'
}

/**
 * Huvudfunktion som söker efter en stad och uppdaterar sidan.
 * Den hämtar koordinater, väderdata och sevärdheter, uppdaterar kartan
 * och genererar dynamiskt väder- och sevärdhetskort.
 *
 * @async
 * @function
 */
async function searchCity() {
  const city = cityInput.value.trim()
  if (!city) return alert("Skriv in en stad!")

  // Hämta koordinater via geokodnings-API
  const coords = await getCoordinates(city)
  if (!coords) {
    alert("Kunde inte hitta staden.")
    return
  }
```

Figur A2: Dokumentation av externa API-funktioner

```
/**
 * Dokumentation för externa API-funktioner:
 *
 * @function getCoordinates
 * @param {string} city - Namn på staden
 * @returns {Promise<{lat: number, lon: number, name: string}>} - Koordinater för staden
 *
 * @function getWeath
 * @param {number} la
 * @param {number} lo
 * @returns {Promise<Object>} - Väderdata
 *
 * @function getPlaces
 * @param {number} lat - Latitud
 * @param {number} lon - Longitud
 * @returns {Promise<Array>} - Lista med sevärdheter i närheten
 *
 * @function getWeatherIcon
 * @param {number} code - Väderkod
 * @returns {string} - Emoji eller SVG för angivet väder
 */
```

interface Object
Provides functionality common to all JavaScript objects.